

Code Review Report

This report should be written at a high level, targeted toward developers familiar with your project. You can assume the reader is familiar with programming topics and with your project but may not have exhaustive knowledge of all areas of the same

This is a technical report, and no formal academic referencing is required. Clearly state each group member's full name, student ID number, and student email address on the first page. Divide the report into sections for each of the points listed below. The expected length is 3-6 pages of text, excluding front-matter, headers, figures, and tables; though this will vary if there are many or few items to report on.

Submit the report as a PDF document through moodle prior to the due date listed on the moodle submission portal. Late submissions will **not be accepted**, please upload regular draft versions of your report (uploaded without clicking the final submit for grading button) to ensure that you do not miss the submission window. Only one member of your group needs to submit the report.

Individual grades will be derived from the group grade, weighted by the peer review completed by other group members.

Report Content

Conduct a structured review of your project codebase and produce a critical evaluation of its structure, design, and development practices. Your review should be grounded in the areas in which you have been working, but must also engage with the project at a whole-system level. This is not a pull-request review: you are not commenting line-by-line on a diff, you are assessing the codebase as an artefact that developers must work in and forming an argued judgement about it.

This review has been divided into 4 sections, described below.

- **Scope and Orientation** (2 *points*). Briefly establish the context for your review: How the project is architected at a high level, which parts of the codebase you have worked in, and which parts of the project your review covers. State explicitly what you did not examine, and why.

Marking Notes: This section is short and simply frames the context of later sections – we want to know where we should expect you to have expert knowledge. A review that claims whole-project insight without evidence of having read the code is less credible than one that says "I worked in these three modules, I have read the build configuration and the public API surface, and I have not examined the front-end".

- **Issues and Recommendations** (13 *points*). Identify **five issues** with the codebase. For each issue, explain what the issue is, referring to the specific files, modules, packages, or subsystems involved. Further explain why you consider it to be an issue, in terms of the concrete consequence for someone developing, testing, extending, or maintaining the project; and explain what you would recommend be done to address it, including what impact that change would have and what it would cost or risk.

Before asserting that something is an oversight, check the project's issue tracker, roadmap, design documents, and contributing guidelines. If the maintainers are already aware of the issue, say so, and engage with their stated position – you can agree with their reasoning or argue why it does not hold.

Marking Notes: Identified issues and associated recommendations should have a moderate scope, with an explanation of what impact making these changes would have. We do not want to call out very minor issues (e.g.

American vs British English spellings, style preferences) or obvious design and library choices that you may disagree with. Individual bugs are out of scope unless you can show that the bug is symptomatic of a structural pattern. Disagreements with language or framework choices will only count where you can demonstrate a specific consequence you actually encountered. Feature requests and "this should be rewritten in X" are not issues.

Issues and positive design choices can draw from the inclusion of required developer processes and tooling, or the lack thereof, where appropriate (e.g. enforced consistent style through external linting). Other areas you may want to consider include module boundaries and coupling between components; the quality of abstractions, whether leaky, absent, or speculative; error handling and failure-mode strategy; testing strategy, including coverage of critical paths and the ability to test components in isolation; build, CI, and release tooling; dependency management and upgrade posture; configuration and environment handling; public API design and stability; contributor-facing documentation and onboarding friction; and how hard the codebase makes it to add the kind of change you were making.

Every claim should be locatable in the repository – provide links to elements in the repository that support your claims.

- **Positive Design Choices** (5 *points*). Identify **three positive design choices** within the project. For each, clearly identify why you consider it to be a positive choice, and give a developer-level explanation of how the choice impacts the developer experience.

Marking Notes: As with the section on the negatives, avoid the obvious. "It has tests" or "it uses a widely adopted framework" is an observation, not an analysis. We are looking for you to identify a specific decision and articulate what it made easy that would otherwise have been hard — ideally something you felt directly while working in the code. Restating the project's own README claims are not sufficient.

- **Prioritization and Synthesis** (5 *points*). If you had the maintainers' attention and could direct their efforts for a single release cycle, in what order should they address your identified issues/recommendations, and why? Rank the issues you identified in the issues and recommendations section and justify that ordering,

weighing impact against cost, risk, and the practical realities of an open-source project (maintainer capacity, backwards compatibility, contributor expectations, etc). Finally, identify which of your recommendations you believe would be hardest to actually land in this particular community, and explain why.

Marking Notes: This section marks judgement, not coverage, and it should not restate the issues identified earlier. We are looking for evidence that you can distinguish what matters most from what is merely true, and that you understand that a technically correct recommendation is not automatically an actionable one. Recognizing the social and process constraints on change in an open-source project, not just the technical ones, is what separates a strong answer here from an adequate one.